GAUSS'S LEMMA AND POLYNOMIAL GCD DOMAINS: AN INFORMAL PROOF FOLLOWING THE UNIT FILES

THIERRY COQUAND

Purpose and how to read this text

This text is written for a test. The claim is that the proof terms of the checker Unit have the same structure as a detailed informal proof. The example is the theorem

if A is a gcd domain, then so is $A[X]$,

together with Gauss's lemma, on which it rests. The development is in the directory `unit/sag/gcddomain`. It has 23 files (plus `All.seg`, which only lists them) and checks in about 0.1 s.

Four levels of detail. We follow van Benthem Jutting [1], who developed a proof in four stages: a proof A "which would normally be given in conversation", very informal and with gaps; a proof B as in "a rather pedantic textbook"; a proof C with more detail, where bars in the margin indicate the context of assumptions and the small lemmas used are noted explicitly; and a proof D, the formal text checked by machine, with a glossary for the human reader. Here:

- proofs A and B are in Part 1;
- proof C is the rest of this text (Parts 2 and 3);
- proof D is the Unit files.

The modules of Unit play the role of Jutting's margin bars: a module `M (x : X) { ...}` fixes the assumptions $x \in X$ for the steps it contains. His explicitly noted lemmas are our named small lemmas (such as `div-eq-right` or `plus-assoc`), and his glossary is the correspondence below. One difference: Jutting refined A into D, while proof C was written from the formal files. The question of the test is how mechanical the passage from C to D is.

Proof C has one section per file, in the order in which the files are checked. Every definition, lemma and step is tagged with the name of the corresponding Unit declaration, written like this: `div-trans`. The reader can therefore put the text and the file side by side. Part 3 (seven files, about 900 lines) contains the algebra. It is written in full: *every* declaration of these files appears in the text. Part 2 describes the files on rings and polynomials more briefly, one paragraph per notion.

The correspondence. The following constructs of Unit have a fixed informal reading.

- A *declaration* `name (x : X) : T = t` is a definition, a lemma or a step of a proof; its parameters are its hypotheses.
- A *module* `module M (x : X) { ...}` is a proof, or a part of a text, under fixed hypotheses: "Let $x \in X$. ...". Its declarations are the named steps of that proof, and `M[a].step` refers to a step of it for the particular value a.
- A *record* is a structure "consisting of ...". A Σ-type $(q : R) \times \ldots$ is "the datum of q such that...". A component of a pair is written `h.1`, `h.2`.

Date: 4 November 2026.

- `split` on a natural number, with the recursive value written `| r`, is a proof by induction; `r` is the induction hypothesis. `split` on a decision $x = 0$ or $x \neq 0$ is a proof by cases. Unit writes a case analysis as its own small declaration, named after what is decided: `step-byP` is "case on whether $c(p) = 0$".
- `calc R { a =< h > b =< k > c }` is a chain of equalities $a = b = c$, each step justified by the term between `=<` and `>`.
- `efq` is "this case is impossible".
- An abbreviation `module D = Domain[A,...]` means "we use the results of the section on `Domain` for this A"; they are then cited as `D.cancel`, etc. The text cites them by their own names.

Some steps of the formal proof would be silent in a usual text. The most frequent is `div-eq-right`: "$d \mid b$ and $b = b'$, hence $d \mid b'$". The text below leaves most of these steps silent, as a usual proof does. They are the main reason why a formal step is often longer than the corresponding sentence.

Foundations. The development uses the base theory of Unit (dependent types, identity types, inductive types and one universe per level). It adds no new axioms. It rests on the parameters of the `Theory` segment of `All.seg`: univalence (`univ`) and replacement (`images`, `surjections`). They are used only to construct $A[X]$ as a quotient of lists of coefficients, and to derive function extensionality.

Contents

Purpose and how to read this text 1
Four levels of detail 1
The correspondence 1
Foundations 2

Part 1. Proofs A and B 2
1. Proof A 2
2. Proof B 2

Part 2. Rings and polynomials 4
3. Logic, identity, functions (`Base.seg`, `Prelude.seg`, `Axioms.seg`, `FunExtUA.seg`) 4
4. Quotients (`Replacement.seg`, `Truncation.seg`, `Quotient.seg`) 4
5. Commutative rings and natural numbers (`Ring.seg`, `NatOrder.seg`) 4
6. Polynomials as lists (`Poly.seg`, `Mul.seg`, `RawLaws.seg`, `RawMultiplication.seg`) 4
7. The ring $A[X]$ (`RingHom.seg`, `PolynomialRing.seg`, `Coefficients.seg`) 5

Part 3. The algebra 5
8. Divisibility and gcd domains (`GcdDomain.seg`) 5
8.1. Divisibility (module `Divisibility`, for a ring A) 5
8.2. Gcd domains 6
8.3. Discrete domains (module `Domain`, for A with `set`, `dec`, `integral`) 6
8.4. Gcds and Lemma M (module `Gcd`, for a gcd domain A) 7
8.5. Discrete fields (module `FieldGcd`) 7
9. The content of a sequence (`Content.seg`) 7
9.1. Bounded sequences (module `Bounded`, for e bounded by n) 8
10. Polynomials over a gcd domain (`PolyBase.seg`) 8
10.1. Division by a scalar 8

11. Gauss's lemma (`Gauss.seg`) 9
11.1. On lists (module `Direct`) 9
11.2. On $A[X]$ (module `Over`) 11
12. Elimination (`Elimination.seg`) 11
13. The degree (`Degree.seg`) 13
14. $A[X]$ is a gcd domain (`PolyGcd.seg`) 13
14.1. The content 13
14.2. The gcd of two non-zero polynomials (module `Main`) 14
14.3. Discrete, non-trivial, integral 15
What a usual text leaves implicit 15
References 15

Part 1. Proofs A and B

1. Proof A

Gauss's lemma: the content of fg is $\mathrm{c}(f)\mathrm{c}(g)$. Dividing by the contents, it is enough to show that a product of two primitive polynomials is primitive. Let d divide all coefficients of fg, and write $f = a + Xf_1$, $g = b + Xg_1$. Then $\gcd(d, a)$ divides all coefficients of $f_1 g$, so by induction it divides $\mathrm{c}(f_1)$, hence all coefficients of f, so it is a unit. In the same way $\gcd(d, b)$ is a unit. Since $d \mid ab$, Euclid's lemma gives $d \mid b$, so d is a unit.

$A[X]$ is a gcd domain. Run Euclid's algorithm on f and g, but multiply by non-zero scalars instead of dividing. This gives D and $\lambda \neq 0$ with $D \mid \lambda f$, $D \mid \lambda g$, and every common divisor of f and g divides D. By Gauss's lemma the primitive part E of D divides f and g, and $\gcd(\mathrm{c}(f), \mathrm{c}(g))\, E$ is a gcd of f and g.

2. Proof B

Let A be a gcd domain: an integral domain with $1 \neq 0$, with a decision of $x = 0$ for every x, and a chosen gcd $\gcd(a, b)$ of every pair. Gcds are determined up to a unit only, and divisibility $a \mid b$ means that $b = aq$ for some q.

Lemma 2.1. *(i) If $d \mid ua$ and $d \mid ub$ then $d \mid u \gcd(a, b)$. (ii) (Euclid) If $\gcd(a, b) \mid 1$ and $a \mid bc$ then $a \mid c$.*

Proof. (i) If $u = 0$ this is clear. Otherwise let $g = \gcd(ua, ub)$. Then $d \mid g$, and $u \mid g$, say $g = ut$. Cancelling u in $ut \mid ua$ and $ut \mid ub$ gives $t \mid a$ and $t \mid b$, so $t \mid \gcd(a, b)$ and $d \mid g = ut \mid u \gcd(a, b)$. (ii) $a \mid ca$ and $a \mid cb$, so $a \mid c \gcd(a, b) \mid c$ by (i). □

The *content* $\mathrm{c}(f)$ of a polynomial f is the gcd of its coefficients, computed as $\gcd(f_n, \gcd(f_{n-1}, \ldots \gcd(f_0, 0)))$. It divides every coefficient of f, and every common divisor of the coefficients of f divides it. If $\mathrm{c}(f) = 0$ then $f = 0$. The polynomial f is *primitive* if every common divisor of its coefficients divides 1. If $\mathrm{c}(f) \neq 0$, then $f = \mathrm{c}(f) f^*$ with f^* primitive. By Lemma 2.1(i) and induction on the number of coefficients: if f is primitive and $d \mid uf_i$ for every i, then $d \mid u$.

Theorem 2.2. *If d divides every coefficient of fg, then $d \mid \mathrm{c}(f)\mathrm{c}(g)$.*

Proof. By induction on the total number N of coefficients of f and g, for all f and g. If $\mathrm{c}(f) = 0$ or $\mathrm{c}(g) = 0$, then $\mathrm{c}(f)\mathrm{c}(g) = 0$ and there is nothing to prove; this covers $N = 0$. Otherwise write $f = \mathrm{c}(f) f^*$ and $g = \mathrm{c}(g) g^*$, with f^* and g^* primitive and with as many coefficients as f and g. Then $fg = \mathrm{c}(f)\mathrm{c}(g)\, f^* g^*$, so it is enough to show that $f^* g^*$ is primitive.

So let $f = a + Xp$ and $g = b + Xq$ be primitive, and let d divide every coefficient of fg. Put $t = \gcd(d, a)$. The coefficient of X^{i+1} in fg is $aq_{i+1} + (pg)_i$. Since $t \mid a$, t divides every coefficient of pg. By the induction hypothesis, applied to p and g (we need the general statement here, since p need not be primitive), $t \mid \mathrm{c}(p)\mathrm{c}(g)$, and $\mathrm{c}(g) \mid 1$, so $t \mid \mathrm{c}(p)$. So t divides a and every coefficient of p, that is, every coefficient of f. Hence $t \mid 1$. In the same way $\gcd(d, b) \mid 1$. Now $d \mid ab$, the constant coefficient of fg, so $d \mid b$ by Lemma 2.1(ii), and $d \mid \gcd(d, b) \mid 1$. □

Corollary 2.3 (Gauss's lemma)**.** *If f is primitive and d divides every coefficient of fg, then d divides every coefficient of g. Hence, if E is primitive, $\lambda \neq 0$ and $\lambda f = Eq$, then $E \mid f$.*

Proof. $d \mid \mathrm{c}(f)\mathrm{c}(g) \mid \mathrm{c}(g)$. For the second statement, λ divides every coefficient of $\lambda f = Eq$, hence of q, so $q = \lambda q'$; then $\lambda f = \lambda E q'$, and $f = Eq'$. □

Theorem 2.4 (elimination)**.** *For $f, g \in A[X]$ there are $D \in A[X]$ and $\lambda \neq 0$ in A with $D \mid \lambda f$, $D \mid \lambda g$, and such that every common divisor of f and g divides D.*

Proof. Let f have degree $< n_f$ and g degree $< n_g$. By induction on $n_f + n_g$. If $n_f = 0$, then $f = 0$, and $D = g$, $\lambda = 1$; the case $n_g = 0$ is symmetric. Otherwise let α and β be the coefficients of f and g of degree $n_f - 1$ and $n_g - 1$. If $\alpha = 0$, f has degree $< n_f - 1$, and we apply the induction hypothesis; likewise if $\beta = 0$. Otherwise, say $n_f \geq n_g$, and let $k = n_f - n_g$. Then $f' = \beta f - \alpha X^k g$ has degree $< n_f - 1$. The induction hypothesis gives (D, λ') for f' and g. Take $\lambda = \lambda'\beta \neq 0$. Then $\lambda f = \lambda' f' + \alpha X^k(\lambda' g)$ and $\lambda g = \beta(\lambda' g)$ are divisible by D, and a common divisor of f and g divides f', hence D. If $n_f < n_g$, exchange f and g. □

Theorem 2.5. *If A is a gcd domain, then so is $A[X]$.*

Proof. $f = 0$ if and only if $\mathrm{c}(f) = 0$, which is decidable. If $fg = 0$, then $\mathrm{c}(f)\mathrm{c}(g) = 0$ by Theorem 2.2, so $f = 0$ or $g = 0$. And $1 \neq 0$ in $A[X]$.

Gcds: if $f = 0$ then g is a gcd, and if $g = 0$ then f is a gcd. Let now $f, g \neq 0$, and take D, λ from Theorem 2.4. Then $D \neq 0$, since $D = 0$ would give $\lambda f = 0$. Write $D = \mathrm{c}(D)E$ with E primitive. If $\lambda f = Dr$, then $\lambda f = E(\mathrm{c}(D)r)$, so $E \mid f$ by Corollary 2.3; likewise $E \mid g$. Let $\gamma = \gcd(\mathrm{c}(f), \mathrm{c}(g))$ and $G = \gamma E$.

$G \mid f$: write $f = Eq$. Then γ divides $\mathrm{c}(f)$, hence every coefficient of Eq, hence every coefficient of q by Corollary 2.3; so $q = \gamma s$ and $f = Gs$. Likewise $G \mid g$.

If $h \mid f$ and $h \mid g$, then $h \mid D$, say $D = ht$, and $h \neq 0$. Write $h = \mathrm{c}(h)h^*$ with h^* primitive. Then $\mathrm{c}(h)$ divides $\mathrm{c}(f)$ and $\mathrm{c}(g)$, hence γ. And $\mathrm{c}(D)E = D = h^*(\mathrm{c}(h)t)$, so $h^* \mid E$ by Corollary 2.3. Hence $h = \mathrm{c}(h)h^* \mid \gamma E = G$. □

Proof B is a usual textbook proof. It treats elements of $A[X]$ as polynomials and does not say how a gcd is chosen. The section *What a usual text leaves implicit*, at the end, lists what proof C adds to it.

Part 2. Rings and polynomials

3. Logic, identity, functions (`Base.seg`, `Prelude.seg`, `Axioms.seg`, `FunExtUA.seg`)

`Base.seg` declares the small data types: the empty type, the unit type `Top` with element `tt`, sums `Plus` with injections `inl`, `inr`, natural numbers, lists, and Boolean operations.

`Prelude.seg` contains the elementary properties of equality, each by one use of the elimination rule J: symmetry `sym`, transitivity `trans`, congruence `ap`, transport `subst`, and congruence of a binary operation `cong2`: $a = a'$ and $b = b'$ give $a \cdot b = a' \cdot b'$. A type is a *proposition* (`isProp`) if any two of its elements are equal. It is a *set* (`isSet`) if all its equality types are propositions. Two pairs are equal when their first components are equal and their second components are

equal up to transport (`Sigma-eq`). A Σ-type of propositions over a proposition is a proposition (`isProp-Sigma`).

`Axioms.seg` states function extensionality and univalence as types (`FunExt`, `Univalence`). It derives their consequences: `funext`, a product of propositions is a proposition (`isProp-Pi`), and a product of sets is a set (`isSet-Pi`). `FunExtUA.seg` derives function extensionality from univalence (HoTT book, §4.9).

4. Quotients (`Replacement.seg`, `Truncation.seg`, `Quotient.seg`)

`Replacement.seg` states the replacement principle of Cavallo and Coquand: the image of a map from a small type into a locally small type is small. `Truncation.seg` defines the propositional truncation as the image of $A \to$ `Top`. `Quotient.seg` constructs, for a proposition-valued equivalence relation $\sim$ on a type P, the quotient $P/\!\sim$ (`Quot`). It consists of

- the class map `inc`, with `relates`: $p \sim q$ gives `inc` $p =$ `inc` q, and `set`: the quotient is a set;
- `induction`: *to prove a proposition for every element of the quotient, it is enough to prove it for the classes* `inc` p;
- `Descend`: a map $P \to S$ into a set S that respects $\sim$ defines a map $P/\!\sim \to S$ (`map`). Its value on `inc` p is the given one (`beta`).

The principle `induction` is used several times below. A mathematician applies it without saying so when reasoning "on a representative".

5. Commutative rings and natural numbers (`Ring.seg`, `NatOrder.seg`)

`Ring.seg` defines a commutative ring (`CRing`) as a structure consisting of a type `R`, elements `z` (0) and `o` (1), operations `add`, `mul`, `neg`, and the laws: associativity (`addA`, `mulA`), commutativity (`addC`, `mulC`), units (`addZ`, `mulO`), inverse (`addN`) and distributivity (`dist`). The module `RL` proves the usual consequences, such as $0x = 0$ (`zero-mul`), $x0 = 0$ (`mul-zero`), $1x = x$ (`one-left`), $u(-v) = -(uv)$ (`mul-neg`), $-0 = 0$ (`neg-zero`), $(x+y)-y = x$ (`add-neg-cancel`), $a(bc) = b(ac)$ (`mul-swap`) and $(ab)(cd) = (ac)(bd)$ (`mul-four`). It also proves the identities on natural numbers used for indices: `plus-zero`, `plus-suc`, `plus-comm`, `suc-inj` and `suc-zero` ($n+1 \neq 0$).

`NatOrder.seg` defines $m \leq n$ (`Le`) and $m < n$ (`Lt`) by recursion. It proves reflexivity, transitivity, the comparison `compare` ($m = n + d$ or $n = m + d$ for some d), and the case distinctions `le-or-gt` and `le-cases`.

6. Polynomials as lists (`Poly.seg`, `Mul.seg`, `RawLaws.seg`, `RawMultiplication.seg`)

A polynomial is represented by the list of its coefficients, the constant term first (`P`). Its *length* is `plen`, and its k-th coefficient `coef` is 0 beyond the length. Two lists are *equivalent* (`peq`) if they have the same coefficients, so trailing zeros do not matter. Sum `addP`, scalar multiple `scal` and product `pmul` are defined by recursion on lists:

$$f \cdot 0 = 0, \qquad f \cdot (b + Xg) = bf + X(fg).$$

Their coefficients are computed in `coef-addP`, `coef-scal` and `coef0-pmul` (the constant coefficient of fg is $f_0 g_0$). Beyond the length every coefficient is zero (`coef-beyond`).

`RawLaws.seg` and `RawMultiplication.seg` prove the ring laws up to `peq`, and that the operations respect `peq`. Among them: `negP` and `neg-coef` (negation), `scal-cong`, `scal-scal` ($a(bf) = (ab)f$), `mul-cons-left` ($(a+Xp)q = aq+X(pq)$), `mul-cong`, `mul-comm`, `mul-scal-left` and `mul-scal-right`.

7. The ring $A[X]$ (`RingHom.seg`, `PolynomialRing.seg`, `Coefficients.seg`)

`RingHom.seg` defines ring homomorphisms; it is needed by `PolynomialRing.seg` for the constant map $A \to A[X]$.

`PolynomialRing.seg` (module `Extension`, for a ring A whose underlying type is a set) defines $A[X]$ as the quotient of lists by `peq` (`Carrier`, with class map `include`). Each operation descends from lists to the quotient because it respects `peq` (`addQ`, `mulQ`, `negQ`, `zeroQ`, `oneQ`). The ring laws of the quotient follow from those on lists by `forall2` and `forall3`: *a proposition about two or three elements of $A[X]$ holds if it holds for lists.* The structure of commutative ring is `ring`. It also defines the constants `constant`, the variable `variable` (X), `constant`$(ab) =$ `constant`(a)`constant`(b) (`constant-mul`), and the lemmas `mul-comm`, `mul-assoc`, `mul-one`.

`Coefficients.seg` (module `Over`) defines the k-th coefficient of an element of $A[X]$. It is obtained by `Descend`, since equivalent lists have the same coefficients (`Coef`, `coef`). On a class it is the coefficient of the list (`coef-include`). The file proves:

- `coef-ext`: *two polynomials with the same coefficients are equal*;
- `law`: an equation between coefficients that holds on lists holds on $A[X]$, since equality of coefficients is a proposition;
- the coefficients of a sum, a scalar multiple, a negative, a product by X, 0 and a constant: `coef-add`, `coef-scale`, `coef-neg`, `coef-variable` (with `shifted` and `shift-coef`), `coef-zero`, `coef-const0` and `coef-constS`;
- `Bounded` d x: every coefficient of x from index d on is zero, that is, x has degree $< d$.

Part 3. The algebra

In this part *every* declaration of the seven files is named. Throughout, A is a commutative ring with underlying type R.

8. Divisibility and gcd domains (`GcdDomain.seg`)

Lemma 8.1 (`empty-prop`, `not-prop`)**.** *The empty type is a proposition. For every type X, $\neg X$ is a proposition.*

Proof. An element of the empty type gives anything. $\neg X = X \to \emptyset$ is a product of propositions (`isProp-Pi`). □

8.1. Divisibility (module `Divisibility`, for a ring A).

Definition 8.2 (`Div`)**.** $a \mid b$ is the datum of q with $b = aq$.

So a proof of $a \mid b$ is a pair (quotient, equation). We never decide whether $a \mid b$.

Lemma 8.3. (1) *(`div-refl`) $a \mid a$, with quotient 1.*
(2) *(`div-zero`) $a \mid 0$, with quotient 0.*
(3) *(`div-one`) $1 \mid a$, with quotient a.*
(4) *(`div-trans`) If $a \mid b$ and $b \mid c$ then $a \mid c$.*
(5) *(`div-mul`, `div-mul-left`) If $a \mid b$ then $a \mid bc$ and $a \mid cb$.*
(6) *(`div-factor`, `div-factor-left`) $a \mid ac$ and $a \mid ca$, with quotient c.*
(7) *(`div-add`, `div-neg`) If $a \mid b$ and $a \mid c$ then $a \mid b + c$; if $a \mid b$ then $a \mid -b$.*
(8) *(`div-scale`) If $a \mid b$ then $ca \mid cb$, with the same quotient.*
(9) *(`div-mul2`) If $a \mid b$ and $c \mid d$ then $ac \mid bd$.*
(10) *(`div-eq-left`, `div-eq-right`) If $a \mid b$ and $a = a'$ (resp. $b = b'$) then $a' \mid b$ (resp. $a \mid b'$).*
(11) *(`div-of-zero`) If $0 \mid b$ then $b = 0$.*

Proof. (4) If $b = ah$ and $c = bk$ then $c = bk = (ah)k = a(hk)$, so the quotient is hk. (5) If $b = ah$, the quotient is hc, since $bc = (ah)c = a(hc)$; for cb, first $cb = bc$. (6) $ac = ac$ by definition; $ca = ac$ by commutativity. (7) If $b = ah$ and $c = ak$ then $b + c = ah + ak = a(h + k)$; and $-b = -(ah) = a(-h)$. (8) $cb = c(ah) = (ca)h$. (9) If $b = ah$ and $d = ck$ then $bd = (ah)(ck) = (ac)(hk)$ (`mul-four`). (10) By transport along the equality. (11) $b = 0 \cdot q = 0$. □

Definition 8.4 (`GcdOf`)**.** A gcd of a and b is the datum of d with $d \mid a$, $d \mid b$, and a function giving $c \mid d$ for every c with $c \mid a$ and $c \mid b$.

A gcd is unique only up to a unit. So it is a datum, not a property.

8.2. **Gcd domains.**

Definition 8.5 (`GcdDomain`)**.** A *(discrete) gcd domain* structure on A consists of

- `set`: R is a set;
- `dec`: for every x, $x = 0$ or $x \neq 0$ (a decision);
- `integral`: if $x \neq 0$ and $y \neq 0$ then $xy \neq 0$;
- `gcd`: a gcd of every pair a, b;
- `nontrivial`: $1 \neq 0$.

8.3. **Discrete domains (module `Domain`, for A with `set`, `dec`, `integral`).**

Lemma 8.6. (1) *(`mul-sub`) $u(a - b) = ua - ub$.*
(2) *(`sub-zero`) If $a - b = 0$ then $a = b$.*
(3) *(`sub-self`) $a - a = 0$.*

Proof. (1) By distributivity and $u(-b) = -(ub)$. (2) $a = (a + b) - b = (b + a) - b = b + (a - b) = b + 0 = b$. (3) This is the inverse law. □

Lemma 8.7 (cancellation, `cancel`)**.** *If $u \neq 0$ and $ua = ub$ then $a = b$.*

Proof. We decide whether $a - b = 0$ (`cancel-cancelD`). If $a - b = 0$, then $a = b$ by Lemma 8.6(2). If $a - b \neq 0$, then $u(a - b) \neq 0$ by integrality. But $u(a - b) = ua - ub = ub - ub = 0$. This case is impossible. □

Lemma 8.8. (1) *(`cancel-zero`) If $u \neq 0$ and $ux = 0$ then $x = 0$.*
(2) *(`div-cancel`) If $u \neq 0$ and $ua \mid ub$ then $a \mid b$, with the same quotient.*
(3) *(`div-prop`) If $a \neq 0$ then $a \mid b$ is a proposition.*
(4) *(`nonzero-prop`) $x \neq 0$ is a proposition.*
(5) *(`mul-nonzero`) If $x \neq 0$ and $y \neq 0$ then $xy \neq 0$.*

Proof. (1) $ux = 0 = u0$; cancel u. (2) If $ub = (ua)q$ then $ub = u(aq)$; cancel u. (3) Two quotients q, q' satisfy $aq = b = aq'$, so $q = q'$ by cancellation. The two equations are then equal, since R is a set. (4) By `not-prop`. (5) This is integrality. □

8.4. **Gcds and Lemma M (module `Gcd`, for a gcd domain A).** We write $\gcd(a, b)$ for the chosen gcd (`gcd`). Its three properties are `gcd-left` ($\gcd(a, b) \mid a$), `gcd-right` ($\gcd(a, b) \mid b$) and `gcd-univ` ($c \mid a$ and $c \mid b$ give $c \mid \gcd(a, b)$).

Lemma 8.9 (Lemma M, `lemma-M`)**.** *If $d \mid ua$ and $d \mid ub$ then $d \mid u \gcd(a, b)$.*

Proof. We decide whether $u = 0$ (`lemma-M-byU`).
If $u = 0$ then $u \gcd(a, b) = 0$, and $d \mid 0$.
If $u \neq 0$ (module `M`, with $u \neq 0$, $d \mid ua$, $d \mid ub$ fixed):

- (`g`) let $g = \gcd(ua, ub)$;
- (`dg`) $d \mid g$, as a common divisor of ua and ub;

- (`ug`) $u \mid g$, since $u \mid ua$ and $u \mid ub$;
- (`t`) let t be the quotient, $g = ut$;
- (`ta`) $t \mid a$: since $ut = g \mid ua$, cancel u (`div-cancel`);
- (`tb`) $t \mid b$, in the same way;
- (`gu`) $g \mid u\gcd(a,b)$: $t \mid \gcd(a,b)$ by `gcd-univ`, hence $ut \mid u\gcd(a,b)$ (`div-scale`), and $ut = g$;
- (`result`) $d \mid g \mid u\gcd(a,b)$. □

Lemma 8.10 (Euclid, `euclid`)**.** *If* $\gcd(a,b) \mid 1$ *and* $a \mid bc$ *then* $a \mid c$.

Proof. $a \mid ca$ and $a \mid cb$ ($cb = bc$), so $a \mid c\gcd(a,b)$ by Lemma 8.9 with $u = c$. And $c\gcd(a,b) \mid c \cdot 1 = c$ (`div-scale`). So $a \mid c$. □

8.5. **Discrete fields (module `FieldGcd`).** Let F be a ring whose type is a set, in which every x is 0 or invertible (`dec`), and where $1 \neq 0$.

Lemma 8.11. (1) *(*`unit-nonzero`*) An invertible element is not* 0.
(2) *(*`decZ`*, by cases* `decZ-decD`*)* $x = 0$ *or* $x \neq 0$.
(3) *(*`integral`*, by cases* `integral-intD`*) If* $x \neq 0$ *and* $y \neq 0$ *then* $xy \neq 0$.
(4) *(*`unit-div`*) An invertible element divides every element.*
(5) *(*`gcd`*, by cases* `gcd-gcdD`*) Every pair has a gcd.*
Hence (`domain`*)* F *is a gcd domain.*

Proof. (1) If $xy = 1$ and $x = 0$ then $1 = 0 \cdot y = 0$. (2) If $x = 0$, done; if x is invertible, then $x \neq 0$ by (1). (3) Decide x. The case $x = 0$ is impossible. If $xw = 1$ and $xy = 0$ then $y = 0$ (`unit-reg`). (4) If $xy = 1$ then $b = 1b = (xy)b = x(yb)$. (5) Decide a. If $a = 0$, then b is a gcd: $b \mid 0 = a$, $b \mid b$, and a common divisor divides b. If a is invertible, then a is a gcd: $a \mid a$, $a \mid b$ by (4), and a common divisor divides a. □

9. The content of a sequence (`Content.seg`)

Let A be a gcd domain (module `Over`). We work with sequences $e \colon \mathbb{N} \to R$.

Definition 9.1.
- (`Below`) A property holds *below* n if it holds for all $i < n$.
- (`Bnd`) e is *bounded by* n if $e_i = 0$ for $i \geq n$. Formally: $e_{n+i} = 0$ for all i.
- (`Prim`) e is *primitive* if every common divisor of the e_i divides 1.
- (`cont`) $\mathrm{c}_0(e) = 0$ and $\mathrm{c}_{n+1}(e) = \gcd(e_n, \mathrm{c}_n(e))$.

Two facts on indices: $n < n+1$ (`lt-top`), and $i < n$ gives $i < n+1$ (`lt-up`).

Lemma 9.2. (1) *(*`cont-div`*)* $\mathrm{c}_n(e) \mid e_i$ *for* $i < n$.
(2) *(*`cont-univ`*) If* $d \mid e_i$ *for all* $i < n$*, then* $d \mid \mathrm{c}_n(e)$.
(3) *(*`cont-M`*) If* $d \mid ue_i$ *for all* $i < n$*, then* $d \mid u\mathrm{c}_n(e)$.

Proof. All three are proved by induction on n.

(1) For $n = 0$ there is no $i < 0$. For $n+1$, let $i < n+1$; then $i < n$ or $i = n$ (`cont-div-step`). If $i < n$: $\mathrm{c}_{n+1}(e) = \gcd(e_n, \mathrm{c}_n(e)) \mid \mathrm{c}_n(e) \mid e_i$, by the induction hypothesis. If $i = n$: $\gcd(e_n, \mathrm{c}_n(e)) \mid e_n$.

(2) For $n = 0$: $d \mid 0$. For $n+1$: $d \mid e_n$, and $d \mid \mathrm{c}_n(e)$ by the induction hypothesis, so d divides their gcd.

(3) For $n = 0$: $d \mid 0 = u \cdot 0$. For $n+1$: $d \mid ue_n$, and $d \mid u\mathrm{c}_n(e)$ by the induction hypothesis. Lemma 8.9 gives $d \mid u\gcd(e_n, \mathrm{c}_n(e))$. □

9.1. Bounded sequences (module `Bounded`, for e bounded by n).

Lemma 9.3. *Let e be bounded by n, and let $\mathrm{c}(e) = \mathrm{c}_n(e)$ (`c`).*

(1) *(`every`) If $d \mid e_i$ for all $i < n$, then $d \mid e_i$ for all i.*
(2) *(`c-div`) $\mathrm{c}(e) \mid e_i$ for all i.*
(3) *(`c-univ`) If $d \mid e_i$ for all i then $d \mid \mathrm{c}(e)$.*
(4) *(`c-zero`) If $\mathrm{c}(e) = 0$ then $e_i = 0$ for all i.*
(5) *(`prim-c`) If e is primitive then $\mathrm{c}(e) \mid 1$.*
(6) *(Corollary, `prim-M`) If e is primitive and $d \mid ue_i$ for all i, then $d \mid u$.*

Proof. (1) Either $n \le i$ or $i < n$ (`every-step`). If $n \le i$, then $i = n + k$ for some k, so $e_i = 0$ and $d \mid 0$. If $i < n$, use the hypothesis. (2) By (1) and Lemma 9.2(1). (3) By Lemma 9.2(2). (4) $0 = \mathrm{c}(e) \mid e_i$, so $e_i = 0$ (`div-of-zero`). (5) $\mathrm{c}(e)$ is a common divisor of the e_i, by (2). (6) $d \mid u\mathrm{c}(e)$ by Lemma 9.2(3), and $u\mathrm{c}(e) \mid u \cdot 1 = u$ by (5). □

10. Polynomials over a gcd domain (`PolyBase.seg`)

Let A be a gcd domain (module `Over`). We write `Pol` for $A[X]$ and `coef` for the coefficients of its elements. For $a \in R$ and $x \in A[X]$, ax (`sc`) is the product of the constant a with x.

Lemma 10.1. (1) *(`coef-sc`) $(ax)_k = ax_k$.*
(2) *(`coef-mul-include`) The coefficients of the product of two classes are those of the product of the lists.*
(3) *(`bnd-list`) The coefficients of a list are bounded by its length.*

Definition 10.2 (`Prim`)**.** $x \in A[X]$ is *primitive* if its sequence of coefficients is primitive. If the class of a list is primitive, so is the list (`prim-include`).

10.1. Division by a scalar.

Construction 10.3 (`divL`, `divL-coef`)**.** Let q be a list and suppose that a divides each coefficient of q. The list q/a is obtained by replacing each coefficient by its quotient, and $q_i = a(q/a)_i$ for every i.

Proof. By induction on the list. The empty list gives the empty list, and $q_i = 0 = a \cdot 0$. For $c + Xq$, the head becomes the quotient of c, and the tail is obtained by the induction hypothesis. □

Let now $a \neq 0$ be fixed (module `Scalar`). For $x \in A[X]$, `Quot` x is the datum of y with $x = ay$, and `Divides` x means that a divides every coefficient of x.

Lemma 10.4. (1) *(`cancel`) If $ay = ay'$ then $y = y'$.*
(2) *(`quot-prop`) `Quot` x is a proposition.*
(3) *(`divide`) If a divides every coefficient of x, there is y with $x = ay$.*

Proof. (1) For every k, $ay_k = (ay)_k = (ay')_k = ay'_k$; cancel a (Lemma 8.7). Then $y = y'$ by `coef-ext`.

(2) Two quotients are equal by (1). The equations are then equal, since $A[X]$ is a set.

(3) Since `Quot` x is a proposition, it is enough to prove it when x is the class of a list q (`induction`). In that case (`divide-list`), a divides each coefficient of the list q (step `w'`). Take y the class of q/a; then $x = ay$ coefficientwise, by `divL-coef`. □

11. Gauss's lemma (`Gauss.seg`)

This is Theorem 2.2 and Corollary 2.3 of proof B. The structure of the proof is as follows.

(1) Lists start with the constant term, so we remove the constant term: $f = a + Xf_1$.

(2) We prove the statement (C) below, one divisibility, and get the other one directly.
(3) The induction is on the total length N, and the statement is the general one (C): it is applied to f_1, which need not be primitive. At each length we first prove the primitive case from (C) at the length below (`PrimStep`), then (C) by dividing by the contents (`Nonzero`). Division by a scalar keeps the length, so no second use of the induction hypothesis is needed.

11.1. **On lists (module `Direct`).** Let A be a gcd domain. In this subsection polynomials are lists.

Definition 11.1.
- (`content`) The content of a list p is $\mathrm{c}(p) = \mathrm{c}_n(p)$, where n is the length of p. Then Lemma 9.3 applies to p (module `At`, with `K`).
- (`Common`) d is a *common divisor* of p if it divides every coefficient of p.
- (`Product`) Statement (C) for p, q: every common divisor of pq divides $\mathrm{c}(p)\mathrm{c}(q)$.

Lemma 11.2. (1) *(`div-sub`) If $d \mid a$ and $d \mid b$ then $d \mid a - b$.*
(2) *(`sub-add`) $(x + y) - x = y$.*

Lemma 11.3 (`tail-common`)**.** *If $d \mid a$ and d is a common divisor of $(a + Xp)q$, then d is a common divisor of pq.*

Proof. Fix i (module `Tail`). Let $x = aq_{i+1}$ (`x`) and $y = (pq)_i$ (`y`).
- (`eq`) $((a + Xp)q)_{i+1} = x + y$, since $(a + Xp)q = aq + X(pq)$;
- (`dx`) $d \mid x$, since $d \mid a$;
- (`dxy`) $d \mid x + y$, by hypothesis and (`eq`);
- (`result`) $d \mid (x + y) - x = y$. □

Definition 11.4 (`Rec`)**.** $\mathrm{Rec}(N)$: statement (C) holds for all p, q of total length N.

Lemma 11.5 (primitive step, `PrimStep`)**.** *Assume* $\mathrm{Rec}(N)$*. Let $a + Xp$ and $b + Xq$ be primitive, of total length $N + 1$, and let d be a common divisor of their product. Then $d \mid 1$.*

Proof. Write $f = a + Xp$ and $g = b + Xq$. First, p and g have total length N (`ef`), and so do q and f (`es`).

Claim (module `One`). Let $a_0 + Xp_0$ be primitive and q_0 primitive, with p_0, q_0 of total length N, and let d be a common divisor of $(a_0 + Xp_0)q_0$. Then $\gcd(d, a_0) \mid 1$.
- (`t`) Let $t = \gcd(d, a_0)$.
- (`ht`) t is a common divisor of $(a_0 + Xp_0)q_0$, since $t \mid d$.
- (`tc`) $t \mid \mathrm{c}(p_0)\mathrm{c}(q_0)$: by Lemma 11.3 ($t \mid a_0$), t is a common divisor of p_0q_0; apply $\mathrm{Rec}(N)$.
- (`ct`) $t \mid \mathrm{c}(p_0)$, since $\mathrm{c}(p_0)\mathrm{c}(q_0) \mid \mathrm{c}(p_0) \cdot 1$, because q_0 is primitive (`prim-c`).
- (`coeffs`) t is a common divisor of $a_0 + Xp_0$: the coefficient of index 0 is a_0, and $t \mid a_0$; the coefficient of index $i + 1$ is $(p_0)_i$, and $t \mid \mathrm{c}(p_0) \mid (p_0)_i$.
- (`result`) $t \mid 1$, since $a_0 + Xp_0$ is primitive.

Now:
- (`cpA`) $\gcd(d, a) \mid 1$, by the claim for a, p, g.
- (`hs`) d is a common divisor of gf, since gf and fg have the same coefficients.
- (`cpB`) $\gcd(d, b) \mid 1$, by the claim for b, q, f.
- (`ab`) $d \mid ab$, the constant coefficient of fg.
- (`db`) $d \mid b$, by Euclid (Lemma 8.10) and (`cpA`).
- (`result`) $d \mid \gcd(d, b) \mid 1$, by (`db`) and (`cpB`). □

Lemma 11.6 (`not-prim-nil`, `primitive`)**.** *The empty list is not primitive. Hence, under* $\mathrm{Rec}(N)$*, the product of two primitive lists of total length $N + 1$ is primitive.*

Proof. 0 is a common divisor of the empty list. If it were primitive, $0 \mid 1$, so $1 = 0$, against `nontrivial`. For the second statement, by cases on the two lists: if one is empty, it is not primitive, so the case is impossible. Otherwise they are $a + Xp$ and $b + Xq$, and we apply Lemma 11.5. □

Lemma 11.7 (primitive part of a list, `Part`)**.** *Let p be a list with $\mathrm{c}(p) \neq 0$, and let $c = \mathrm{c}(p)$ (`c`). Let $p^* = p/c$ (`qs`). Then*

(1) *(`length`) p^* has the same length as p (`div-len`);*
(2) *(`eq`, `scaled`) $p_i = cp_i^*$ for every i, so p is equivalent to cp^*;*
(3) *(`prim`) p^* is primitive.*

Proof. (1) Division replaces each coefficient and keeps the list structure (by induction, `div-len`). (2) By `divL-coef`. (3) Let d be a common divisor of p^*. Then $cd \mid cp_i^* = p_i$ for every i, so $cd \mid c = c \cdot 1$ (`c-univ`). Cancel $c \neq 0$: $d \mid 1$. □

Lemma 11.8 (non-zero contents, `Nonzero`)**.** *Assume* $\mathrm{Rec}(N)$*. Let p, q have total length $N + 1$ and non-zero contents. Then (C) holds for p, q.*

Proof. Take the primitive parts p^*, q^* (`Pp`, `Pq`), and let $u = \mathrm{c}(p)\mathrm{c}(q)$ (`u`).

- (`len'`) p^* and q^* have total length $N + 1$, by Lemma 11.7(1).
- (`prim`) p^*q^* is primitive, by Lemma 11.6.
- (`scaled`) pq is equivalent to $u(p^*q^*)$:

$$pq \sim (\mathrm{c}(p)p^*)(\mathrm{c}(q)q^*) \sim \mathrm{c}(p)(p^*(\mathrm{c}(q)q^*)) \sim \mathrm{c}(p)(\mathrm{c}(q)(p^*q^*)) \sim u(p^*q^*).$$

- (`result`) Let d be a common divisor of pq. Then $d \mid (pq)_i = u(p^*q^*)_i$ for every i. Since p^*q^* is primitive, $d \mid u$ (`prim-M`). □

Theorem 11.9 (statement (C), `product`)**.** *For all lists p, q, every common divisor of pq divides* $\mathrm{c}(p)\mathrm{c}(q)$*.*

Proof. By induction on the total length N (`induction`).

$N = 0$ (`at-zero`): p is empty, so $\mathrm{c}(p)\mathrm{c}(q) = 0$ and $d \mid 0$. (If p were not empty, its length would be positive.)

$N + 1$ (`step`), with the induction hypothesis $\mathrm{Rec}(N)$: decide whether $\mathrm{c}(p) = 0$ (`step-byP`). If $\mathrm{c}(p) = 0$ then $\mathrm{c}(p)\mathrm{c}(q) = 0$, and $d \mid 0$. Otherwise, decide whether $\mathrm{c}(q) = 0$ (`step-byQ`). If $\mathrm{c}(q) = 0$, likewise. Otherwise apply Lemma 11.8. □

Lemma 11.10 (`multiply-common`)**.** *If c is a common divisor of p and d a common divisor of q, then cd is a common divisor of pq.*

Proof. By induction on q. For the empty list, pq is empty. For $b + Xq$, $p(b + Xq) = bp + X(pq)$. The coefficient i of bp is $p_i b$, and $cd \mid p_i b$ (`div-mul2`). The coefficients of $X(pq)$ are 0 and those of pq, which cd divides by the induction hypothesis (`multiply-common-shift-common`). □

Corollary 11.11 (multiplicativity of the content)**.** $\mathrm{c}(pq) \mid \mathrm{c}(p)\mathrm{c}(q)$ *(`content-mul-left`, by Theorem 11.9), and* $\mathrm{c}(p)\mathrm{c}(q) \mid \mathrm{c}(pq)$ *(`content-mul-right`, by Lemma 11.10 and `c-univ`).*

Corollary 11.12 (Gauss's lemma on lists, `gauss`)**.** *If p is primitive and d is a common divisor of pq, then d is a common divisor of q.*

Proof. $d \mid \mathrm{c}(p)\mathrm{c}(q)$ by Theorem 11.9, and $\mathrm{c}(p)\mathrm{c}(q) \mid \mathrm{c}(q) \cdot 1$ since $\mathrm{c}(p) \mid 1$. So $d \mid \mathrm{c}(q) \mid q_i$. □

11.2. **On $A[X]$ (module `Over`).** Corollary 11.12 is restated as `gauss-list`.

Theorem 11.13 (Gauss's lemma, `GaussAt`)**.** *Let $d \neq 0$. If $x \in A[X]$ is primitive and d divides every coefficient of xy, then d divides every coefficient of y.*

Proof. Let `Concl` y be the conclusion, and `Stmt` $x\,y$ the whole implication. Since $d \neq 0$, each $d \mid y_j$ is a proposition (`div-prop`), so `Concl` y (`concl-prop`) and `Stmt` $x\,y$ (`stmt-prop`) are propositions. By `forall2` it is enough to prove the statement for classes of lists (`on-lists`). For these the coefficients are those of the lists, and we apply `gauss-list`. □

Remark 11.14. A usual text would not have the hypothesis $d \neq 0$. It is needed only to pass from lists to $A[X]$. For $d = 0$, a proof of $0 \mid 0$ may have any quotient, so $0 \mid y_j$ is not a proposition, and the argument "it is enough to prove it for lists" does not apply. Where $d = 0$ is needed (integrality, below), the statement on lists is used directly.

Lemma 11.15 (division by a primitive, `DivPrim`)**.** *Let E be primitive, $\lambda \neq 0$ and $\lambda f = Eq$. Then $E \mid f$.*

Proof. (With `S` the division by λ.)

- (`dq`) λ divides every coefficient of q: it divides every coefficient of $\lambda f = Eq$; apply Theorem 11.13.
- (`q'`) So $q = \lambda q'$ for some q'.
- (`same`) $\lambda f = Eq = E(\lambda q') = \lambda(Eq')$.
- (`result`) $f = Eq'$, by cancelling λ. □

12. Elimination (`Elimination.seg`)

This is Theorem 2.4 of proof B: Euclid's algorithm in $A[X]$, where we multiply by a non-zero scalar instead of dividing.

Lemma 12.1 (two ring identities, module `Ident`)**.** *In every commutative ring:*

(1) *(`ident`)* $(LB)f = L(Bf - Tg) + T(Lg)$*;*
(2) *(`ident2`)* $(LB)g = B(Lg)$.

Proof. (1) $L(Bf - Tg) + T(Lg) = (L(Bf) + L(-(Tg))) + T(Lg) = (L(Bf) - T(Lg)) + T(Lg) = L(Bf) + (-(T(Lg)) + T(Lg)) = L(Bf) + 0 = L(Bf) = (LB)f$. (2) $(LB)g = L(Bg) = B(Lg)$. □

Let now A be a gcd domain (module `Over`), and write `Bounded` n x for "x has degree $< n$" (`Bounded`).

Definition 12.2 (`Elim`)**.** An *elimination datum* for $f, g \in A[X]$ consists of $D \in A[X]$ (`gd`) and $\lambda \in A$ (`lam`) with $\lambda \neq 0$ (`lam-nz`), $D \mid \lambda f$ (`div-f`), $D \mid \lambda g$ (`div-g`), and such that every common divisor of f and g divides D (`common`).

An elimination datum for f, g is one for g, f (`swap`).

Lemma 12.3 (`sc-one`, `zero-left`)**.** $1x = x$*. If f has degree < 0 then $(g, 1)$ is an elimination datum for f, g.*

Proof. $f = 0$, since all its coefficients are 0 (`f0`, by `coef-ext`). Then $g \mid 0 = 1f$, $g \mid g = 1g$, and a common divisor of f, g divides g. □

Lemma 12.4 (indices and powers of X)**.** (1) *(`plus-assoc`)* $(a + b) + c = a + (b + c)$.

(2) *(`xp`)* $X^k x$ *is defined by recursion on k.*
(3) *(`coef-xp`)* $(X^k x)_{k+i} = x_i$.
(4) *(`xp-mul`)* $X^k x = (X^k \cdot 1)x$.

(5) *(*`bound-down`*) If x has degree $< n+1$ and $x_n = 0$, then x has degree $< n$.*

Proof. (1)–(4) By induction on a, resp. k. (5) The coefficient $n+0$ is $x_n = 0$, and the coefficient $n + (i+1) = (n+1) + i$ is 0 by hypothesis. □

The identities of `Ident` are used in the ring $A[X]$ (`IB`).

Definition 12.5 (`Rec`)**.** Rec(N): for all f, g and n_f, n_g with $n_f + n_g = N$, f of degree $< n_f$ and g of degree $< n_g$, there is an elimination datum for f, g.

Lemma 12.6 (the elimination step, `Reduce`)**.** *Assume* Rec(N). *Let f have degree $< n'_f + 1$ and g degree $< n'_g + 1$, with $n'_f = k + n'_g$ and $n'_f + (n'_g + 1) = N$. Assume that the top coefficient $g_{n'_g}$ is non-zero. Then f, g have an elimination datum.*

Proof. Let $\alpha = f_{n'_f}$ (`alpha`), $\beta = g_{n'_g}$ (`beta`), $T = \alpha X^k$ (`T`), and

$$f' = \beta f - Tg \qquad (\texttt{f'}).$$

- (`coef-T`) $(Tg)_j = \alpha(X^k g)_j$.
- (`coef-f'`) $f'_j = \beta f_j - \alpha(X^k g)_j$.
- (`idx`) $(X^k g)_{n'_f + i} = g_{n'_g + i}$, since $n'_f + i = k + (n'_g + i)$.
- (`top`) $f'_{n'_f} = \beta\alpha - \alpha\beta = 0$.
- (`above`) $f'_{n'_f + j + 1} = \beta \cdot 0 - \alpha \cdot 0 = 0$, by the bounds on f and g.
- (`bounded`) So f' has degree $< n'_f$, by (`top`) and (`above`).
- (`common'`) A common divisor of f and g divides $\beta f - Tg = f'$.
- (`rec`) Since $n'_f + (n'_g + 1) = N$, Rec(N) gives an elimination datum (D, λ') for f', g.
- (`lam`) Let $\lambda = \lambda'\beta$.
- (`sc-f`) $\lambda f = \lambda' f' + T(\lambda' g)$, by `ident`.
- (`sc-g`) $\lambda g = \beta(\lambda' g)$, by `ident2`.
- (`result`) (D, λ) is an elimination datum for f, g: $\lambda \neq 0$ since $\lambda' \neq 0$ and $\beta \neq 0$; $D \mid \lambda' f'$ and $D \mid \lambda' g$, so D divides λf and λg; and a common divisor of f, g divides f' and g, hence D. □

Lemma 12.7 (both bounds positive, `Tops`)**.** *Assume* Rec(N). *Let f have degree $< n'_f + 1$ and g degree $< n'_g + 1$, with $(n'_f + 1) + (n'_g + 1) = N + 1$. Then f, g have an elimination datum.*

Proof. Let $\alpha = f_{n'_f}$ and $\beta = g_{n'_g}$. The arithmetic: $n'_f + (n'_g + 1) = N$ (`eF`), $(n'_f + 1) + n'_g = N$ (`eG`) and $n'_g + (n'_f + 1) = N$ (`eSym`).

Decide whether $\alpha = 0$ (`byF`). If $\alpha = 0$, f has degree $< n'_f$ (`bound-down`); apply Rec(N) with (`eF`).

Otherwise decide whether $\beta = 0$ (`byG`). If $\beta = 0$, g has degree $< n'_g$; apply Rec(N) with (`eG`).

Otherwise compare n'_f and n'_g (`byCmp`). If $n'_f = n'_g + d$, apply Lemma 12.6 with $k = d$. If $n'_g = n'_f + d$, apply Lemma 12.6 to g, f (with (`eSym`) and $\alpha \neq 0$), and exchange (`swap`). □

Theorem 12.8 (elimination, `elimination`)**.** *If f has degree $< n_f$ and g has degree $< n_g$, then f, g have an elimination datum.*

Proof. We prove Rec(N) by induction on N (`elim`), and apply it with $N = n_f + n_g$.

$N = 0$ (`onZero`): then $n_f = 0$, and Lemma 12.3 applies. (n_f cannot be a successor, since $n_f + n_g = 0$.)

$N + 1$: by cases on n_f (`onF`). If $n_f = 0$, apply Lemma 12.3. If $n_f = n'_f + 1$, by cases on n_g (`onG`). If $n_g = 0$, apply Lemma 12.3 to g, f and exchange. If $n_g = n'_g + 1$, apply Lemma 12.7. □

13. The degree (Degree.seg)

Let A be a gcd domain (module `Over`). To apply Theorem 12.8 we need a bound on the degree of an element of $A[X]$. A list gives a bound, its length, but different lists for the same polynomial give different lengths. So we use the *exact* bound, which is unique.

Definition 13.1.
- (`TopF`) For $n = 0$ the condition is empty (`Top`); for $n + 1$ it is $e_n \neq 0$.
- (`DegF`) A *degree* of e is the datum of n such that e is bounded by n and `TopF` n e.

So the degree in this sense is the usual degree plus one, and 0 for the zero polynomial.

Lemma 13.2. (1) *(*`top-eq`*,* `top-prop`*)* `TopF` n e *is a proposition.*
(2) *(*`bnd-prop`*) "e is bounded by n" is a proposition.*
(3) *(*`top-vs-bound`*) If e is bounded by m, then* `TopF` n e *fails for $n = m + d + 1$.*
(4) *(*`unique-step`*,* `unique-cmp`*) Two degrees of e are equal.*
(5) *(*`deg-prop`*) The type of degrees of e is a proposition.*

Proof. (1) `Top` has one element, and a negation is a proposition. (2) A product of equalities in a set. (3) $n = m + d + 1$ is not 0; and $e_{m+d} = 0$ contradicts $e_{m+d} \neq 0$. (4) Let n and m be degrees. Compare them (`unique-cmp`): say $n = m + d$ (`unique-step`). If $d = 0$, then $n = m$. If d is a successor, (3) gives a contradiction. The other case is symmetric. (5) By (4), the two first components are equal; the second components are equal, by (1) and (2). □

Construction 13.3 (degree of a list, `deg-list`)**.** Every list has a degree.

Proof. By induction on the list. The empty list has degree 0. For $c + Xp$, let n be the degree of p (`cons-deg`). If $n = m + 1$, then $c + Xp$ has degree $m + 2$. If $n = 0$, decide whether $c = 0$ (`byC`). If $c = 0$, then $c + Xp$ has degree 0 (`zero-cons`). Otherwise it has degree 1. □

Definition 13.4 (`deg`, `dg`, `dg-bnd`)**.** Every $x \in A[X]$ has a degree (`deg`): since this is a proposition (Lemma 13.2(5)), it is enough to give it for lists (`induction`). We write $\deg x$ for the number (`dg`). x is bounded by $\deg x$ (`dg-bnd`). We let $\mathrm{c}(x) = \mathrm{c}_{\deg x}(x)$, for which Lemma 9.3 applies (module `Cx`, with `K`).

Lemma 13.5 (`zero-of-coefs`)**.** *If all coefficients of x are 0, then $x = 0$.*

Proof. By `coef-ext`. □

14. $A[X]$ is a gcd domain (PolyGcd.seg)

Let A be a gcd domain (module `Over`). For $x \in A[X]$, $\mathrm{c}(x)$ (`content`) is the content of the coefficients of x, bounded by $\deg x$.

14.1. The content.

Lemma 14.1. (1) *(*`zero-content`*) If $\mathrm{c}(x) = 0$ then $x = 0$.*
(2) *(*`content-zero`*) If all coefficients of x are 0 then $\mathrm{c}(x) = 0$.*
(3) *(*`zero-coef`*) If $x = 0$ then all coefficients of x are 0.*
(4) *(*`zero-mul-coef`*) If $h = 0$ then all coefficients of hr are 0.*
(5) *(*`nz-div`*) If $h \mid f$ and $\mathrm{c}(f) \neq 0$ then $\mathrm{c}(h) \neq 0$.*
(6) *(*`shuffle`*) $(cE)q = E(cq)$.*

Proof. (1) By `c-zero` and `zero-of-coefs`. (2) 0 divides all coefficients, so $0 \mid \mathrm{c}(x)$. (3) The coefficients of 0 are 0. (4) $hr = 0 \cdot r = 0$. (5) If $\mathrm{c}(h) = 0$ then $h = 0$ by (1); then $f = hr$ has all coefficients 0 by (4), and $\mathrm{c}(f) = 0$ by (2). (6) Associativity and commutativity. □

Lemma 14.2 (primitive part, `PP`)**.** *Let* $\mathrm{c}(x) \neq 0$*, and let* $c = \mathrm{c}(x)$ *(*`c`*; Lemma 9.3 applies as* `K`*, and* `S` *is the division by c). There is* x^* *(*`xs`*) with* $x = cx^*$ *(*`eq`*), and* x^* *is primitive (*`prim`*).*

Proof. c divides every coefficient of x (`c-div`), so $x = cx^*$ (`divide`). Let d be a common divisor of x^*. Then $cd \mid cx_i^* = x_i$ for every i, so $cd \mid c = c \cdot 1$ (`c-univ`). Cancel c: $d \mid 1$. □

14.2. **The gcd of two non-zero polynomials (module `Main`).** Let f, g have non-zero contents.

(1) (`el`) Let (D, λ) be an elimination datum for f, g (Theorem 12.8, with the bounds $\deg f$ and $\deg g$); write `lam` for λ.
(2) (`nD`) $\mathrm{c}(D) \neq 0$. Otherwise $D = 0$, so $\lambda f = Dr = 0$, so $\lambda f_i = 0$ and $f_i = 0$ for every i ($\lambda \neq 0$, `cancel-zero`). Then $\mathrm{c}(f) = 0$, a contradiction.
(3) (`PD`, `E`, `cD`) Write $D = \mathrm{c}(D)E$ with E primitive (Lemma 14.2).
(4) (`ef`) $\lambda f = E(\mathrm{c}(D)r)$, where $\lambda f = Dr$: indeed $Dr = (\mathrm{c}(D)E)r = E(\mathrm{c}(D)r)$ (`shuffle`). (`eg`) Likewise $\lambda g = E(\mathrm{c}(D)s)$.
(5) (`Ef`, `Eg`) $E \mid f$ and $E \mid g$, by Lemma 11.15.
(6) (`gam`) Let $\gamma = \gcd(\mathrm{c}(f), \mathrm{c}(g))$. (`ngam`) $\gamma \neq 0$, since $\gamma \mid \mathrm{c}(f) \neq 0$.
(7) (`Gp`) Let $G = \gamma E$.

Lemma 14.3 (`Part`)**.** *Let* $x \in A[X]$ *with* $E \mid x$*, say* $x = Eq$*, and* $\gamma \mid \mathrm{c}(x)$*. Then* $G \mid x$*.*

Proof.
- (`dgi`) γ divides every coefficient of Eq: it divides $\mathrm{c}(x)$, which divides every coefficient of $x = Eq$.
- (`s`) By Gauss's lemma (Theorem 11.13, E primitive, $\gamma \neq 0$), γ divides every coefficient of q, so $q = \gamma s$.
- (`result`) $x = Eq = E(\gamma s) = (\gamma E)s = Gs$. □

Lemma 14.4 (`Common`)**.** *If* $h \mid f$ *and* $h \mid g$ *then* $h \mid G$*.*

Proof.
- (`hD`) $h \mid D$, by the elimination datum.
- (`nh`) $\mathrm{c}(h) \neq 0$, by Lemma 14.1(5).
- (`PH`, `ch`) Write $h = \mathrm{c}(h)h^*$ with h^* primitive (Lemma 14.2).
- (`ch-div`) If $h \mid x$ then $\mathrm{c}(h)$ divides every coefficient of x: $x = hr = \mathrm{c}(h)(h^*r)$.
- (`ch-gam`) $\mathrm{c}(h) \mid \gamma$: $\mathrm{c}(h)$ divides $\mathrm{c}(f)$ and $\mathrm{c}(g)$, by (`ch-div`) and `c-univ`.
- (`eD`) $\mathrm{c}(D)E = D = ht = (\mathrm{c}(h)h^*)t = h^*(\mathrm{c}(h)t)$.
- (`hsE`) $h^* \mid E$, by Lemma 11.15 with $\lambda = \mathrm{c}(D) \neq 0$.
- (`cC`) $\mathrm{c}(h) \mid \gamma$ also holds for the constants in $A[X]$.
- (`result`) $h = \mathrm{c}(h)h^* \mid \gamma E = G$ (`div-mul2`). □

(`gcdOf`) Hence G is a gcd of f and g: $G \mid f$ and $G \mid g$ by Lemma 14.3 (with $\gamma \mid \mathrm{c}(f)$, resp. $\gamma \mid \mathrm{c}(g)$), and every common divisor divides G by Lemma 14.4.

Theorem 14.5 (`gcdP`)**.** *Any two* $f, g \in A[X]$ *have a gcd.*

Proof. Decide whether $\mathrm{c}(f) = 0$ (`byF`). If so, $f = 0$, and g is a gcd. Otherwise decide whether $\mathrm{c}(g) = 0$ (`byG`). If so, $g = 0$, and f is a gcd. Otherwise apply `Main`. □

14.3. **Discrete, non-trivial, integral.**

Lemma 14.6. (1) *(*`decP`*, by cases* `byDec`*)* $x = 0$ *or* $x \neq 0$*.*
(2) *(*`nontrivialP`*)* $1 \neq 0$ *in* $A[X]$*.*

Proof. (1) Decide whether $\mathrm{c}(x) = 0$. If so, $x = 0$ (Lemma 14.1(1)). If not, $x \neq 0$, since $x = 0$ would give $\mathrm{c}(x) = 0$ (Lemma 14.1(3),(2)). (2) The constant coefficient of 1 is 1, and that of 0 is 0. □

Lemma 14.7 (integrality, `integralP`)**.** *If $x \neq 0$ and $y \neq 0$ then $xy \neq 0$.*

Proof. Let `Integral` $x\,y$ be the statement. It is a proposition, so it is enough to prove it for classes of lists p, q (`forall2`, `integral-lists`).

- (`list-content-nz`) A non-zero list has non-zero content: if $\mathrm{c}(p) = 0$ then all coefficients of p are 0, so its class is 0.
- Suppose $pq = 0$. Then 0 divides every coefficient of pq, so $0 \mid \mathrm{c}(p)\mathrm{c}(q)$ by Theorem 11.9, that is, $\mathrm{c}(p)\mathrm{c}(q) = 0$. This contradicts integrality of A, since $\mathrm{c}(p) \neq 0$ and $\mathrm{c}(q) \neq 0$. □

Theorem 14.8 (`domain`)**.** *If A is a gcd domain, then $A[X]$ is a gcd domain.*

Proof. $A[X]$ is a set (`set`), and it is discrete (`decP`), integral (`integralP`), has gcds (`gcdP`) and $1 \neq 0$ (`nontrivialP`). □

What a usual text leaves implicit

A comparison of proof C with proof B shows the steps that the formal proof makes explicit and an informal proof usually omits.

(1) *Gcds and quotients are data.* A divisibility $a \mid b$ is a quotient with an equation. A gcd is a chosen element with its properties. This is why $\mathrm{c}(fg)$ and $\mathrm{c}(f)\mathrm{c}(g)$ are compared by divisibility both ways, and never by equality.

(2) *Reasoning on a representative.* $A[X]$ is a quotient of lists. Each time a statement is proved "on the lists", the text checks that it is a proposition (`quot-prop`, `stmt-prop`, `deg-prop`, `integralP`). This is why Theorem 11.13 assumes $d \neq 0$. It is also why the degree must be the exact one, and not the length of some list.

(3) *The induction hypothesis* of Gauss's lemma is the general statement (C), not the primitive case (Section 11, point 3).

(4) *Cases on decisions.* Every "if $x = 0 \ldots$, otherwise $\ldots$" uses the decision `dec` of the gcd domain. It is written as its own small declaration (`lemma-M-byU`, `step-byP`, `byF`, ...).

(5) *Rewriting.* Steps such as "$d \mid b$ and $b = b'$, so $d \mid b'$" (`div-eq-right`) and index arithmetic ($n'_f + (n'_g + 1) = N$, `eF`) appear as explicit steps.

References

[1] L. S. van Benthem Jutting, *The development of a text in AUT-QE.* Preprint, Technological University Eindhoven; in P. Braffort (ed.), Proceedings of the Symposium on APL, Paris, December 1973.